DESIGN DOCUMENT · HLD + LLD

AI Inference Battlefield

An LLM serving platform built to be attacked, plus an AI SRE that defends it

`Go · Gin · Python · LangGraph · MCP · Ollama · React · Kubernetes · Prometheus`

88% → 0% errors under a 10× spike	**3 s** to eject a degraded GPU	**5%** exposure of a bad model	**0%** errors after the AI SRE's fix

Author: Kalyan Dahake · October 2026 · all results measured on a single CPU-only workstation

Contents

1 Executive summary

2 Problem and goals

3 High-level design (HLD)

4 LLD — AI Gateway

5 LLD — Simulated GPU worker

6 LLD — Rollout controller and edge fleet

7 LLD — AI SRE agent

8 Observability

9 Deployment and security

10 Missions and measured results

11 Lessons learned

12 Operations, limitations and roadmap

1 Executive summary

AI Inference Battlefield is an LLM serving platform designed to be attacked. A load generator and chaos engine run realistic production incidents against a Kubernetes-hosted inference fleet. Each attack is measured with defenses off and on. The final stage hands operations to an AI SRE agent, built on an open-source model, that detects, diagnoses, remediates (with human approval) and verifies against user-facing SLOs.

88% → 0% errors under a 10× spike	**3 s** to eject a degraded GPU	**5%** exposure of a bad model	**4 / 50** edge devices hit by a broken rollout

What it demonstrates

- **Inference traffic engineering:** admission control, backpressure, load/KV/prefix-aware routing, outlier ejection, circuit breakers, budgeted retries, fallback, multi-tenant fair queueing.
- **Progressive delivery for models:** offline eval gate, canary with metric health gates and automatic rollback, canary capacity scaling, ring rollouts to an edge fleet.
- **Resilience at the edge:** devices that keep serving through network partitions and sync buffered telemetry.
- **Agentic operations done safely:** a small local LLM inside a fixed state machine, with an allowlist, argument validation, policy limits, human approval and SLO-based verification.
- **Engineering honesty:** every number is measured, and the document records the bugs the attacks found in the defenses themselves.

Technology

Layer	Choice
Gateway, rollout controller	Go 1.27, Gin, Prometheus client_golang
GPU workers, attacker, edge fleet, evals	Python 3.12, FastAPI, aiohttp
AI SRE	LangGraph 1.2, Model Context Protocol (MCP) 1.30, langchain-ollama, Ollama
Model	qwen2.5:3b (agent), qwen2.5:0.5b (optional real serving), on CPU
Console	React 19, Vite 7, TypeScript
Platform	Kubernetes 1.37 on kind, Prometheus 3.5, Grafana 12

2 Problem and goals

LLM inference fails differently from ordinary web services. Requests are long (seconds), expensive, and heterogeneous: a 6,000-token prompt costs about 50× a chat turn in KV-cache memory. GPUs fail *grey*: they get slow, not dead. A model release can be fast and wrong. GPU utilization says little about user experience. Generic L7 load balancing (round robin, HTTP health checks) is the wrong tool.

Goals

1. Build the smallest system that reproduces the real failure modes of LLM serving, on a laptop without GPUs.
2. For each failure, measure the naive behaviour, then the behaviour with a specific defense, using the same load.
3. Make releases safe: no bad model should reach more than a small, measurable fraction of users or devices.
4. Give the whole system to an AI agent and keep it safe: humans approve, code enforces policy, users' SLOs decide success.

Non-goals

- Absolute GPU performance numbers (GPUs are simulated; dynamics are the point).
- High availability of the control plane (single replicas with persisted state).

Service level objectives

SLO	Target	Measured where
User-visible error ratio	≤ 2%	client side (attacker) and gateway
p99 time to first token (TTFT)	≤ 2 s	gateway, short windows for verification
Shed traffic	explicit 429/503 in milliseconds, never a timeout	gateway

3 High-level design (HLD)

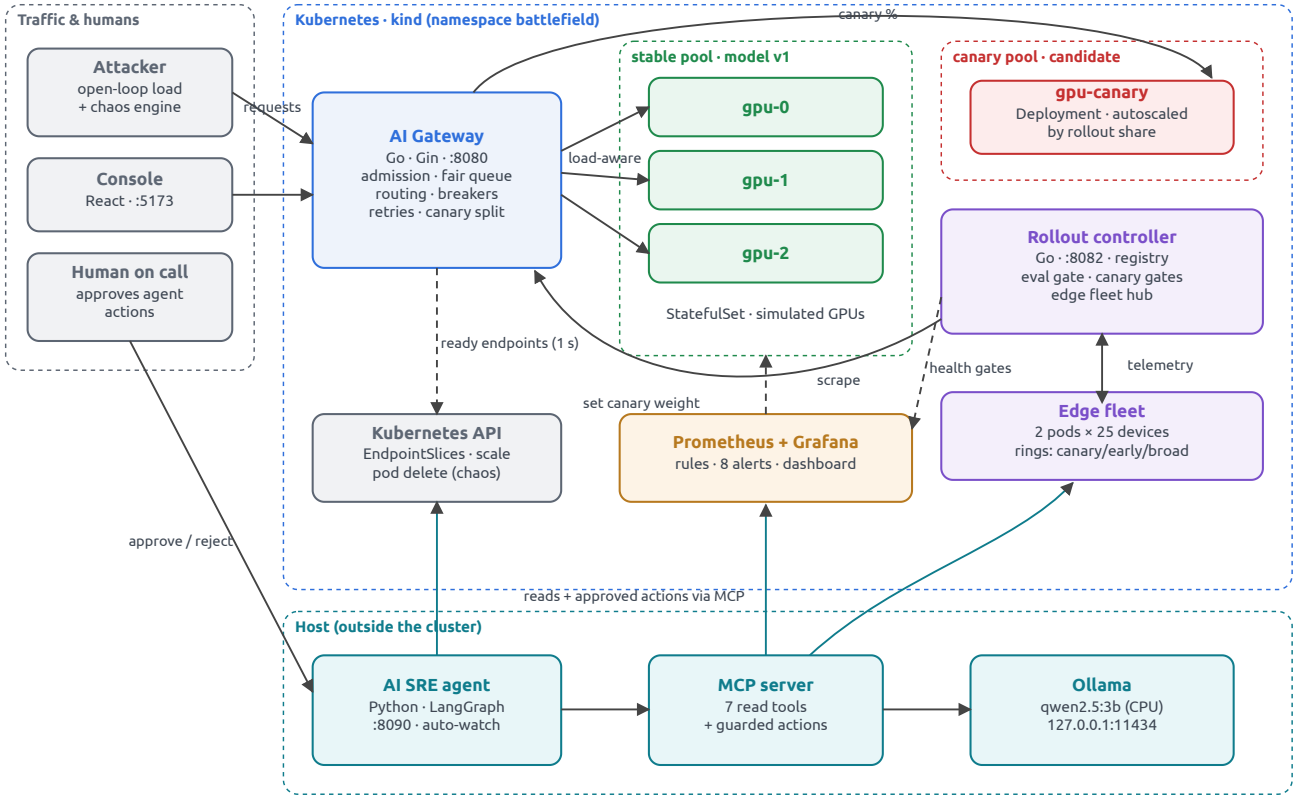


Figure 1 — High-level architecture. Solid arrows are request/command paths; dashed arrows are observation paths.

Components

Component	Responsibility	Interfaces
AI Gateway (Go)	Only path users → GPUs. Ten defenses that can each be switched at runtime, canary split, tenant quotas, persisted state.	`POST /v1/generate`, `/admin/*`, `/metrics`
GPU worker (Python)	One pod = one simulated GPU (slots, KV budget, prefill/decode); optional Ollama proxy; fault injection.	`/v1/generate`, `/state`, `/chaos`, `/metrics`
Attacker (Python)	Open-loop load scenarios; client-side measurement; pod kills; edge chaos fan-out.	`/attacks`, `/chaos/*`
Rollout controller (Go)	Registry + eval gate, canary steps with Prometheus gates, canary autoscaling, edge fleet hub with ring rollouts.	`/registry`, `/rollouts`, `/fleet/*`
Edge fleet (Python)	50 devices: local inference, bounded store-and-forward telemetry, versions pulled by ring.	`/devices`, `/chaos/*`
Eval harness (Python)	Golden set against a pinned version; posts accuracy, p99, cost per 1k tokens.	CLI → `/registry/:v/eval`
AI SRE (Python)	LangGraph incident graph; MCP tools; human approval; SLO verification; auto-watch.	`/incidents`, `/digest`
Console (React)	Missions, live battlefield, attack and defense controls, rollouts and fleet, incident approval.	same-origin proxy to all APIs

Component	Responsibility	Interfaces
Prometheus + Grafana	Pod-annotation scraping, recording rules, 8 alerts, generated 22-panel dashboard.	:9090, :3000

Key flows

1. **Data path:** attacker/console → gateway → (admission → router → attempt) → GPU → streamed tokens.
2. **Discovery path:** gateway lists ready EndpointSlices every 1 s and polls each GPU's `/state` every 500 ms.
3. **Release path:** eval → controller sets canary weight on the gateway → Prometheus gates → promote or roll back.
4. **Edge path:** device → telemetry batch → hub returns desired version for its ring → device installs.
5. **Operations path:** Prometheus alerts + digest → agent → MCP → human approval → action → SLO verification.

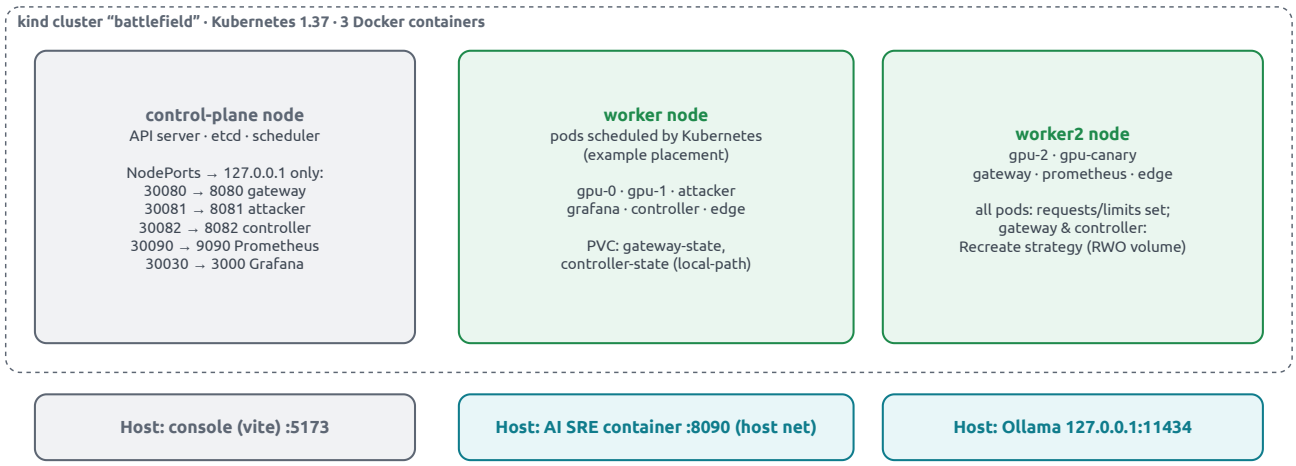


Figure 2 — Deployment topology. Nothing is reachable from the LAN: every port binds to 127.0.0.1.

4 LLD — AI Gateway

Packages: `config` (defenses, tenants, store), `admission`, `router`, `backend` (registry, discovery, polling, breaker), `stats` (per-second ring for the console), `metrics`, and `cmd/gateway` (HTTP, proxy, retries, fallback, persistence).

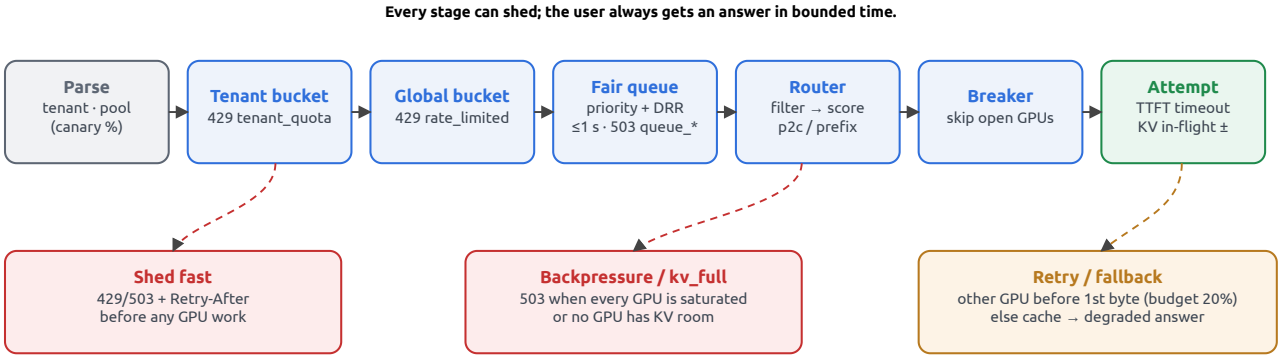


Figure 3 — Gateway request pipeline (hardened mode).

4.1 Admission control

- **Order matters:** tenant token bucket first, then the global bucket. If the global bucket came first, a noisy tenant's rejected traffic would still burn shared tokens.
- **Bounded in-flight** (`max_inflight` = total GPU slots, 96). Beyond that, a bounded queue (200) with a deadline (1 s hardened).
- **Scheduling:** best priority class first (0 interactive, 1 standard, 2 batch), then deficit round robin across tenants weighted by share. When the queue is full, the newest waiter of a strictly lower priority is evicted.
- Every outcome is explicit: `tenant_quota`, `rate_limited`, `queue_full`, `queue_timeout`, `evicted_by_priority`.

4.2 Routing

Eligibility filter (in order): right pool · not excluded by an earlier attempt · not manually ejected · **not gone from EndpointSlices** (membership, applied even with every defense off) · weight > 0 · healthy* · not outlier-ejected* · breaker not open* · KV fits* · long-prompt split* (* = when that defense is on).

```
score(gpu) = ( (inflight + queue_depth) / slots
             + 0.5 * kv_util                 # when kv_aware
             + 0.25 * ttft_ewma_ms / 300
             + 2 * error_ewma ) / weight

kv_fits   = max(reported kv_used, gateway kv_inflight) + request_kv <= kv_budget
saturated = every eligible GPU has inflight >= 2*slots or queue >= slots   -> 503 backpressure
```

Mode	Algorithm	Use
round_robin	weighted rotation	baseline that fails Attack #2
least_loaded	min score	small fleets
p2c	power of two random choices on score	default hardened: O(1), avoids herding

Mode	Algorithm	Use
prefix	consistent hashing on session or prompt[:256], 64 virtual nodes per GPU, bounded load ≤ 1.25 × mean	prefix/KV cache reuse

Long-prompt pool: with ≥ 3 GPUs, the last GPU by name is reserved for prompts ≥ 8,000 chars. Short requests may borrow it while its KV utilization is below 30%.

4.3 Health

- **Polling:** `/state` every 500 ms (400 ms timeout); 2 failures → unhealthy.
- **Discovery:** EndpointSlices via the Kubernetes API every 1 s (read-only RBAC); DNS fallback outside a cluster. A backend that leaves the list but still has requests in flight is marked *draining* and never routed to again.
- **Outlier ejection:** each second, per pool: eject for 15 s if TTFT > 3× the pool's lower median and > 800 ms, or the error EWMA is above 0.5, provided another GPU remains.
- **Overload ≠ breakage:** only connection errors, 500 and 502 count as hard failures; 503/504 are load signals. TTFT EWMAs ignore prompts over 2,000 chars, so prefill doesn't look like sickness.

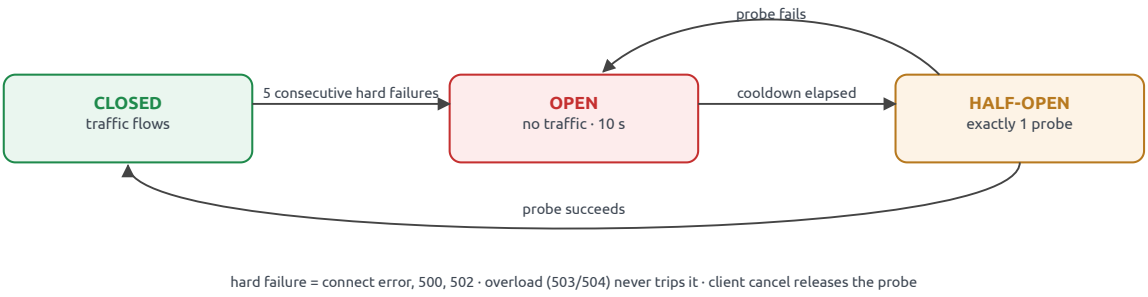


Figure 4 — Per-GPU circuit breaker.

4.4 Retries, fallback and persistence

- Up to 3 attempts, each on a different GPU, only before the first byte reaches the client. Retry budget: retries over the last 10 s ≤ 20% of requests + 3.
- Attempt timeout covers time to first token (6 s hardened); a stream may then run until it finishes.
- Fallback: canary pool empty → stable pool; otherwise the last good answer for the same prompt from a 2,000-entry LRU cache; otherwise a degraded 200 marked `X-Fallback`.
- State (defenses, tenants, canary weight, per-GPU weights and ejections keyed by pod name) is written atomically to `/data/gateway.json` on a PersistentVolume after every admin change and restored at startup.

4.5 Defense presets

Setting	naive	hardened
admission / backpressure	off	on
rate limit / burst	120 / 60	150 / 75
in-flight / queue / deadline	96/200/2s	96/200/1s
routing	round_robin	p2c
kv_aware, long_prompt_pool, outlier_ejection	off	on
health_checks, circuit_breaker, retries, fallback	off	on
attempt timeout	30 s	6 s
tenant_quotas	off	on

4.6 Tenants

Tenant	Rate	Burst	Wt	Priority
default	150	75	2	standard
acme	60	30	3	interactive
globex	60	30	2	standard
batch-jobs	40	20	1	batch

4.7 Gateway API

Method & path	Purpose
`POST /v1/generate`	inference: `{prompt, max_tokens, stream, tenant, session}`; headers `X-Tenant`, `X-Model-Version`; response headers `X-Node`, `X-Pool`, `X-Attempts`, `X-Fallback`
`GET /admin/state`	defenses, every backend snapshot with score, canary weight, admission queue, tenants
`GET /admin/stats?seconds=N`	per-second rps, ok/shed/errors/fallback, p50/p99 latency, p99 TTFT, traffic per pool and GPU
`GET /admin/events`	ejections, breaker transitions, discovery, config changes (last 500)
`POST /admin/defenses` · `/defenses/preset/:name`	partial JSON merge, or the naive / hardened preset
`POST /admin/tenants`	create or update a tenant quota, weight and priority
`POST /admin/backends/:gpu/{weight|eject|restore}`	operator overrides, persisted by pod name
`POST /admin/canary`	canary traffic share 0..1 (used by the rollout controller)
`POST /admin/chaos/:gpu[/reset|/crash]`	fault injection forwarded to one GPU or `all`
`GET /metrics` · `/healthz`	Prometheus metrics, liveness

5 LLD — Simulated GPU worker

Each pod models one GPU serving one model. The gateway cannot tell it apart from a real inference server: it streams Server-Sent Events, enforces capacity, and reports the same signals (slots, queue depth, KV utilization, TTFT, tokens/s).

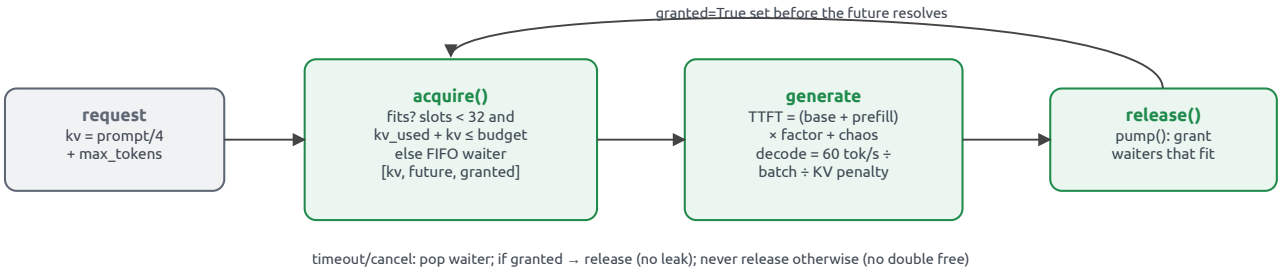


Figure 5 — Simulated GPU: slot and KV-cache accounting.

```
est_tokens(text)      = len(text) / 4
kv(request)           = prompt_tokens + max_tokens                (budget 65,536 tokens)
prefill_s             = prompt_tokens / 4000  (x0.2 on prefix-cache hit; LRU of 2,048 prefixes)
ttft_s                = (0.120 + prefill_s) * LATENCY_FACTOR + chaos.slow_ttft  (x U[0.9, 1.1])
decode_tok_per_s      = 60 * decode_factor / (1 + 0.02*(active-1)) / kv_penalty / LATENCY_FACTOR
kv_penalty            = 1 + (kv_util - 0.85) * 12   when kv_util > 0.85   (preemption/swapping)
```

Slot accounting (race-safe)

A waiter is `[kv, future, granted]`. `pump()` sets `granted` and takes the slot *before* resolving the future. On timeout or cancellation the waiter is removed; if `granted` is set, the slot is released. This fixed a leak (timeout racing a grant). The first fix had caused double releases, because `wait_for` cancels the future. `worker/test_slots.py` races 500 timeouts and 100 cancellations against grants and asserts exact accounting.

Fault injection

Fault	API	Effect
Slow GPU	`{"slow_ttft_ms": 4000}`	adds latency before the first token
Slow decode	`{"decode_factor": 0.3}`	slower tokens/s
Errors	`{"error_rate": 0.5}`	500s
KV squeeze	`{"kv_budget": 6000}`	tiny KV cache
Hang	`{"hang": true}`	requests and `/state` hang; `/ready` fails
Crash	`POST /chaos/crash`	process exits; restarted

Model versions

Version	Latency ×	Accuracy	$/1k tok
v1 stable	1.0	0.95	0.10
v2 bad	1.8	0.85	0.14
v3 good	1.0	0.96	0.09

> Synthetic mode answers eval questions with an "oracle" that is right with probability ORACLE_ACCURACY, so model quality is measurable without a real model. Real mode (`MODE=real`) proxies to Ollama.

6 LLD — Rollout controller and edge fleet

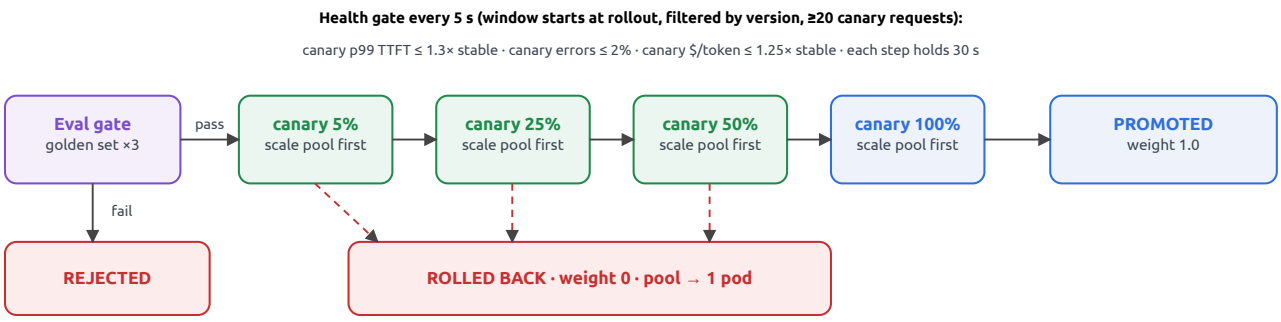


Figure 6 — Model rollout state machine (rollout controller).

6.1 Eval gate

The harness pins traffic with `X-Model-Version` (the gateway maps versions to pools from what GPUs report; unknown versions return 404). It runs 40 questions × 3 and posts accuracy, p50/p99 and cost per 1k tokens. Gate: accuracy ≥ 0.90, accuracy drop ≤ 3 points vs baseline, p99 ≤ 1.3× baseline, cost ≤ 1.25× baseline.

6.2 Canary health gates

```
window = clamp(time since rollout start, 15s, 60s)        # no samples from earlier candidates
canary_requests = sum(increase(worker_requests_total{version="<cand>"}[window]))      # must be >= 20
ttft_p99(v)     = histogram_quantile(0.99, sum by (le)(rate(worker_ttft_seconds_bucket{version="<v>"}[window])))
cost_tok(v)     = rate(worker_cost_dollars_total) / (rate(output_tokens) + rate(prompt_tokens))
error_ratio     = rate(gw_requests_total{pool="canary",outcome="error"}) / rate(gw_requests_total{pool="canary"})
```

NaN/Inf from Prometheus is sanitized to 0 (Go's JSON encoder rejects NaN). Before each step the controller scales `gpu-canary` to `max(1, ceil(weight × stable_replicas × 1.2))` and waits for readiness. RBAC allows only get/patch on the scale subresource of `gpu` and `gpu-canary`.

6.3 Edge fleet

- Ring = `sha1(device_id) mod 100`: <10 canary, <40 early, else broad (4 / 17 / 29 devices).
- Each device infers every 2 s and uploads every ~4 s (batches of ≤ 200). Buffer is capped at 300 records; the oldest is dropped and counted. Records older than 15 s on arrival are counted as late.
- A device is online if seen in the last 12 s. Ring rollout: set the ring's desired version → hold 20 s → judge only online devices already on the new version (≥ 2) → error rate > 5% halts and points every ring back to the previous version.
- Chaos: partition a fraction of devices for N seconds, add uplink latency and loss.

6.4 Persistence

Registry (with evals) and fleet desired versions are snapshotted every 2 s when changed (atomic rename) to `/data/controller.json`. A rollout in progress is not resumed after a restart.

7 LLD — AI SRE agent

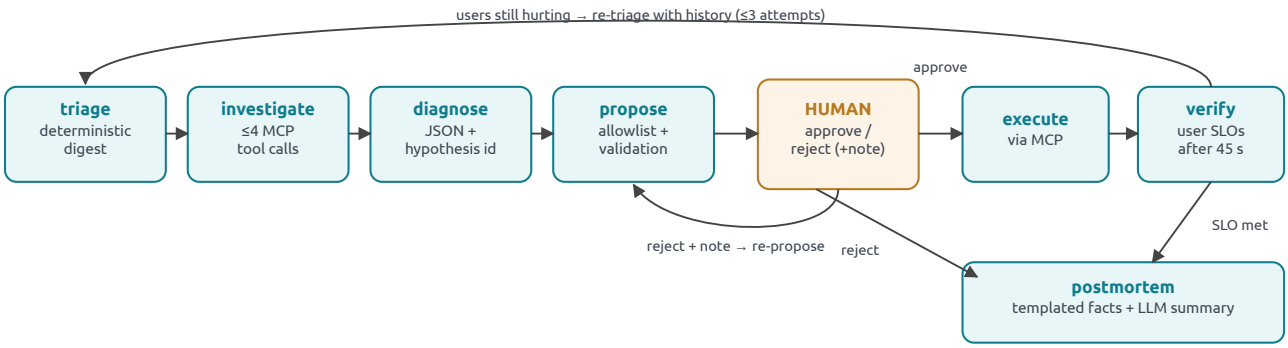


Figure 7 — AI SRE incident graph (LangGraph).

7.1 Why this shape

A 3B model on CPU is good at reading evidence and unreliable at choosing arguments. The graph is fixed; the model makes bounded decisions inside each node; every LLM step has a deterministic fallback; and all safety lives in code.

7.2 Evidence digest (triage)

Plain Python turns gateway state, Prometheus and the controller into ~20 lines of facts plus ranked hypotheses, each with a suggested action. Examples: GPU TTFT > 3× fleet median → `shift_traffic(gpu, 0)`; errors > 5% with admission off → `enable_defense(admission)`; one tenant > 60% of traffic with others shed → `set_tenant_rate`; canary TTFT > 1.3× stable → `set_canary_weight(0)`; edge ring errors > 10% → `fleet_rollback`. Injected-chaos labels and attacker events are hidden; events older than 10 minutes are dropped.

7.3 MCP tools and guarded actions

Read tools (MCP)	Allowlisted actions (validated)
fleet_health · traffic_summary · firing_alerts · recent_events · rollout_status · k8s_pods · pod_logs · evidence_digest	shift_traffic · eject_backend · restore_backend · set_canary_weight (decrease only) · scale_gpus (3–6) · restart_pod (GPU pods only) · set_tenant_rate · enable_defense · fleet_rollback · no_action

Validation: typed arguments, known targets, **no-op detection** (e.g. weight already 1), **policy** (the agent can never increase canary traffic), and no repeat of an action that already failed. A rejected model proposal falls back to the highest-ranked untried hypothesis.

7.4 Verification

```
kpis(window=15s)  : gateway per-second stats + 30s client-probe error ratio
errors            = max(user_error_ratio, gateway_error_ratio)
improved          = (errors_after <= 1% or <= 0.7 x before) and (ttft_after <= 1.5s or <= 0.7 x before)
                    and not (errors_after > 1.2 x before + 1%)
within_slo        = errors_after <= 2% and p99_ttft_after <= 2s
next              = within_slo ? postmortem : (attempts < 3 ? triage : postmortem)
status            = resolved | mitigated (better, SLO breached) | unresolved | rejected
```

7.5 Operation

- **Auto-watch:** every 15 s; opens an incident after 2 consecutive checks with a high-severity hypothesis or firing alert; skips while an incident is open or within 120 s of the last one.
- **Human loop:** LangGraph `interrupt()` pauses at approval; the console resumes it. A rejection with a note re-enters `propose` with that feedback (≤ 2 times).
- **Postmortem:** impact, root cause, evidence, actions table and follow-ups are generated from data; the LLM writes only a 2–3 sentence summary (template fallback). 18 s end to end.
- Runs on the host network so it can reach Ollama on 127.0.0.1; the API binds to 127.0.0.1:8090.

8 Observability

Metrics (selected)

Source	Metrics
Gateway	gw_requests_total{outcome,code,pool,tenant} · gw_shed_total{reason,tenant} · gw_ttft_seconds · gw_request_seconds · gw_retries_total · gw_fallbacks_total · gw_backend_* (healthy, breaker, weight, ejected, inflight, ttft) · gw_canary_weight · gw_defense_enabled
Worker	worker_ttft_seconds · worker_request_seconds · worker_queue_depth · worker_active_requests · worker_kv_utilization · worker_tokens_per_second · worker_cost_dollars_total · worker_prefix_cache_hits_total
Attacker (users)	attacker_requests_total{scenario,tenant,outcome} · attacker_latency_seconds
Controller / edge	rollout_canary_weight · rollout_outcomes_total · model_eval_* · fleet_devices · fleet_ring_error_rate · fleet_telemetry_late_total · edge_buffered_records

Alerts

Alert	Condition
HighTTFT	stable p99 TTFT > 1.5 s for 30 s
GPUSlow	GPU TTFT EWMA > 3× fleet median and > 0.8 s
KVCachePressure	KV utilization > 90% for 20 s
GPUUnreachable	gateway cannot poll a GPU for 10 s
HighErrorRate	pool error ratio > 2% for 20 s
HeavyLoadShedding	> 20% shed for 30 s
TenantStarvation	a tenant has > 50% of its requests shed
CanaryRegression	canary p99 TTFT > 1.3× stable while canary weight > 0

Per-backend gauges are reset every second so pods that no longer exist stop alerting. The Grafana dashboard is generated from code (`deploy/observability/gen_grafana.py`): users, gateway, GPU fleet, rollout and tenants rows.

9 Deployment and security

Resource	Kind	Notes
gpu	StatefulSet (3) + headless Service	stable names gpu-0..2 for targeted chaos; readiness /ready, liveness /health
gpu-canary	Deployment (1–4) + headless Service	scaled by the controller
gateway	Deployment (1, Recreate) + NodePort 30080 + PVC	SA: list EndpointSlices
controller	Deployment (1, Recreate) + NodePort 30082 + PVC	SA: get deployments/statefulsets, patch the two scale subresources
attacker	Deployment + NodePort 30081	SA: get/list/delete pods (chaos)
edge	Deployment (2 × 25 devices)	talks to the controller hub
prometheus / grafana	Deployments + NodePorts	SA: get/list/watch pods, endpoints, services

- **Least privilege:** namespaced Roles only, scoped to the verbs and (where possible) resource names each component needs.
- **No LAN exposure:** every host port binds to 127.0.0.1; Ollama stays on loopback.
- **Images:** Go binaries on distroless/static (nonroot); Python on python:3.12-slim; pinned dependency versions.
- **Platform note:** the host runs cgroup v1, so the kind config sets `failCgroupV1: false` for Kubernetes 1.37.

10 Missions and measured results

#	Attack	Defenses off	Defenses on
02	1000 req/s spike (10× capacity)	88% errors, p99 8.9 s	**0% errors**, p99 2.3 s, fast 429s
03	GPU TTFT → 4 s	p99 20 s, 10% errors	ejected in **3 s**, 0% errors
04	6k-token prompt flood	21% errors, p99 24 s	0% errors; short prompts p50 0.92 s; only kv_full shed
05a	GPU pod killed	19% errors (DNS lag)	**0%** even with defenses off (EndpointSlices)
05b	GPU process hangs	2.6% errors, p99 unbounded	0% errors, p99 2.7 s
06	Tenant floods 500 req/s	others 15–22% success	others **100%**; noisy tenant at quota
07	Bad model (+80% latency, −12% accuracy, +40% cost)	—	eval gate rejects; if bypassed, rollback at **5%**; a good v3 promotes 5→100%
08	40% of edge devices partitioned	—	keep serving; **280 late records synced, 0 lost**
09	Broken edge model rollout	—	halted at the canary ring: **4 / 50** devices
10	Silent GPU degradation, nobody paging	—	agent pages itself, 2 attempts, **0% errors, p99 TTFT 0.8 s**

Final-boss timeline (Mission 10)

t	Event
0 s	gpu-1 silently gains +4 s TTFT; defenses are all off
121 s	auto-watch opens an incident; the model diagnoses gpu-1 and proposes draining it; human approves
+45 s	verification: users **not** better (errors 0% → 5.8%: the remaining GPUs are overloaded) → re-triage
	model proposes sending 100% of traffic to the canary → **refused by policy** → fallback: enable admission control
+45 s	errors 5.8% → 0%, p99 TTFT 7.0 s → 0.8 s, 16% shed → **resolved within SLO**; templated postmortem with follow-ups

11 Lessons learned

#	What the measurement showed	Change
1	Breakers and ejection treated long-prompt prefill and capacity timeouts as failures, ejected busy GPUs, and cascaded load (goodput 21%).	Separate overload (503/504) from breakage; TTFT EWMAs from short prompts only.
2	Global limit before tenant quotas let a noisy tenant starve others.	Tenant quota first, then global.
3	An eval of “v3” silently ran on v1 and passed.	Route by reported version; unknown version → 404.
4	A good canary was rolled back at 50%: one pod, half the traffic.	Scale canary capacity with traffic share.
5	Health gates mixed in the previous candidate's samples.	Version-filtered queries, window from rollout start.
6	NaN from Prometheus broke the status API (empty 200).	Sanitize NaN/Inf.
7	DNS caching kept a dead pod in rotation for 30 s+.	EndpointSlice membership, applied regardless of defenses.
8	Slots leaked on a timeout/grant race; the first fix double-released.	Explicit grant flag + race test.
9	Draining a sick GPU without admission control made users worse; verifier counted alerts.	Verify on user SLOs; retry loop; mitigated ≠ resolved.
10	Agent proposed 100% traffic to an untested model; test harness approved it.	Policy: agent may only reduce canary weight.
11	qwen3:4b spent 14 min reasoning, no postmortem.	Template facts; LLM writes only the summary (18 s).

12 Operations, limitations and roadmap

Run

```
make up            # kind cluster + build + load + deploy
make console       # http://127.0.0.1:5173
make agent         # AI SRE (local Ollama)
make spike DUR=60  # baseline | ramp | long-prompts | noisy-tenant
make chaos-slow GPU=gpu-2 ; make kill-pod GPU=gpu-1 ; make pod-killer
make defenses-naive | defenses-hardened
make eval && make canary-start ; make fleet-rollout VERSION=edge-v2-bad
make test          # Go unit tests (race-clean) + worker slot-accounting race test
python3 scripts/experiment.py --defenses hardened --scenario spike --duration 40
```

Limitations

- GPUs are simulated; real-token mode exists but isn't load-tested on CPU.
- Single-replica gateway and controller (state persisted, not HA).
- qwen2.5:3b on CPU: ~2 minutes from page to proposal; sometimes wrong targets (guardrails and the human catch them).
- Host on cgroup v1 (kind workaround in place).

Roadmap

- Shared gateway state (Redis/etcd) and multiple gateway replicas.
- Real serving engine (vLLM) behind the same worker contract on a GPU node; real KV metrics.
- Autoscaling the stable pool from queue depth/TTFT (KEDA) and letting the agent propose capacity plans.
- Agent evaluation suite: inject each mission N times, score diagnosis accuracy and time to mitigate per model.
- Tracing (OpenTelemetry) from gateway to GPU for per-request breakdowns.